塔克创新
XTARK INNOVATION

塔克串口助手

使用手册

V2.3

更多资讯·欢迎关注塔克创新微信公众号

烟台塔克电子科技有限公司

版权申明

本手册版权归属塔克创新所有，并保留一切权力，受法律保护。非经(书面形式)同意，任何单位及个人不得擅自摘录或修改本手册部分或全部内容，违者我们将追究其法律责任。

塔克媒体

塔克官网	www.xtark.cn
淘宝店铺	https://xtark.taobao.com
塔克哔哩	https://space.bilibili.com/511052131
销售邮箱	sales@xtark.cn

塔克淘宝，微信公众号二维码

更多产品·欢迎关注塔克创新淘宝店铺

更多资讯·欢迎关注塔克创新微信公众号

本手册为塔克创新自主研发的串口调试助手使用说明手册，软件不断更新，最新软件和手册请关注“塔克创新”公众号获取。

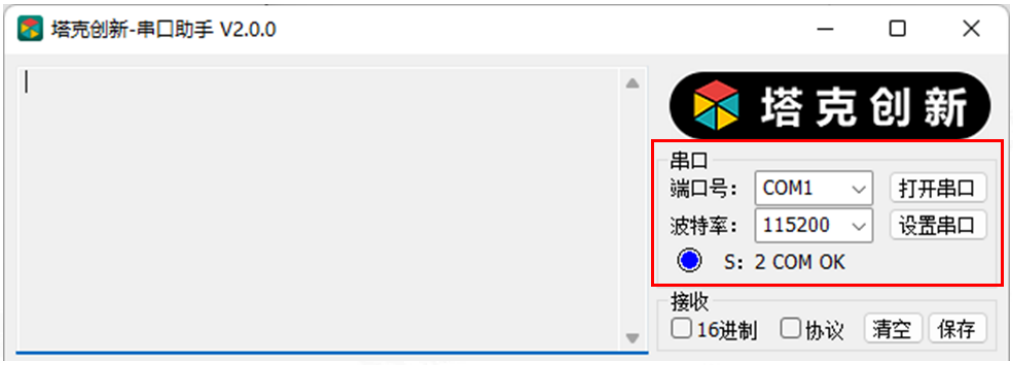
1. 产品介绍

串口调试助手软件为塔克创新自主开发的一款用于智能车、机器人制作的串口软件，在一般串口调试助手基础上增加控制和协议发送功能，均使用统一的 X-Protocol 协议。控制功能可用于机器人智能车控制调试，滑块非常适合机械臂舵机控制。协议发送功能，适合控制命令调试和参数调试。

2. 使用说明

2.1. 串口设置

使用软件前，首先设置串口，具体如下图所示。



- 软件启动后，会自动搜索可用的串口，并在端口号下拉框处显示可用串口。如果有可用串口单击“打开串口”按钮即可打开对应串口，并且显示串口状态和参数信息（S:COM3_ON 115200 N 8 1），串口参数为默认参数（波特率：115200，校验位：NONE，数据位：8，停止位 1）。

- 端口号下拉框可选择和切换不同串口。

- 波特率下拉框可以选择不同波特率。

- 设置串口按钮可以打开不常用的“校验位”“数据位”“停止位”串口参数设置对话框。

- 进入串口参数设置对话框可进行参数设置，完成后单击参数更新即可完成参数设置。考虑到常用情况，参数设置不具有记忆性，每次进入均显示常用的默认参数值。

- 不支持串口热插拔，热插拔会报错误，建议关闭串口后再进行串口插拔。

2.2. 接收说明

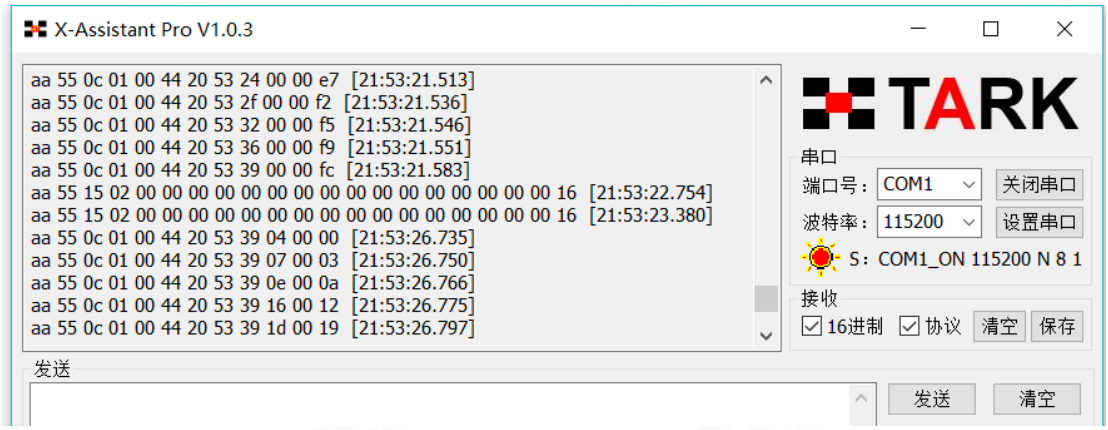
- 串口接收的数据可实现 ASCII 和 16 进制显示，默认为 ASCII 显示。

- 16 进制显示，勾选“16 进制”复选框后之后再接收数据将显示 16 进制的字节。取消勾选“16 进制”复选框后，再接收的数据将显示 ASCII 字符数据。

- 清空按钮可以清除接收窗口显示的内容，并且清除接收字节计数。

- 保存按钮可以保存接收内容为 txt 格式文件，文件名默认为“xx 月 xx 日 xx 时 xx 分 xx 秒_.txt”，其中 xx 为当前时间，“_”后可增加内容字符描述。

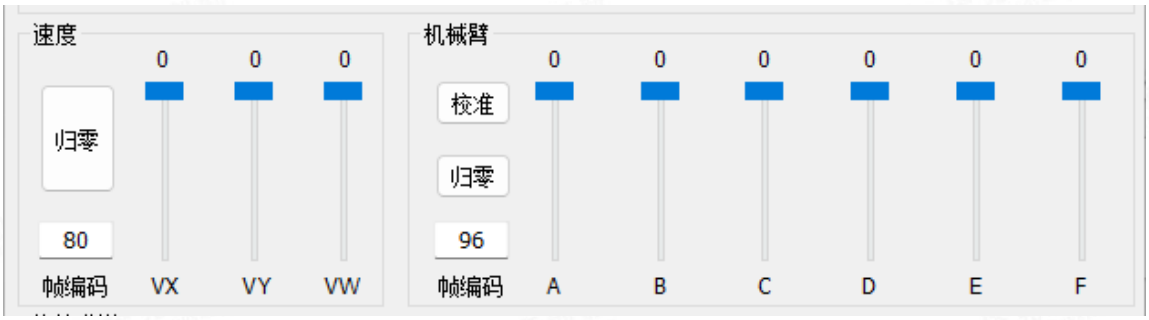
- 在 16 进制模式下，协议复选框，将按照 X-Protocol 协议进行数据接收，每帧数据显示 1 行，并增加时间戳。效果如下，非常适合不同帧内容的数据传输系统分析。



2.3. 发送说明

- 可以在发送区，发送您发送的任意字符。
- 支持 16 进制发送，勾选“16 进制”复选框后，发送的时候将对发送区的内容进行 16 进制和字符互转。
- 支持定时发送，可以自定义发送周期。
- 清除按钮可以清除发送区的内容，并且清除发送字节计数。

2.4. 控制功能



速度控制

“速度”栏可以实现机器人速度控制，基于塔克 X-Protocol 协议，帧 ID 是 80（0x50）。

- 速度控制，通过滑块发送矢量控制速度，包含 X 轴、Y 轴和 W 旋转速度。
- 滑块 VX 为 X 轴速度，滑块 VY 为 Y 轴速度，滑块 VW 为旋转速度。

- 鼠标拖动滑块则发送连续发送控制帧。
- 归零按键，可快速归零，发送零速度帧。
- 数据传输协议，采用 X-Protocol 协议
- 帧定义：AA-55 | 0B | XX | XX-XX XX-XX XX-XX | XX
- 帧解释：帧头 | 帧长 | 帧码 | 滑块 VX 滑块 VY 滑块 VW | 校验和
- 帧长度：12 个字节，固定值 0x0B，包括 5 个帧控制位和 6 个数据位
- 帧编码：可用户自定义，默认 0x50(80)，范围 0~255
- 滑块：三轴速度数据，XY 轴范围±2000,单位 m/s 放大(1000 倍)，W 轴范围±6280，单位弧度/s(放大 1000 倍)
- 校验和：前面数据累加和的低 8 位

机械臂控制

“机械臂”栏滑块可以实现机械臂舵机角度控制，基于塔克 X-Protocol 协议，帧 ID 是 96（0x60）。

- 机械臂控制，通过滑块发送六轴机械臂数据。
- 滑块 A~F 代表，6 个舵机。
- 鼠标拖动滑块则发送连续发送控制帧。
- 归零按键，可快速归零，发送 0 度角度帧。
- 校准按键，发送校准指令帧，帧编码 0x6F，数据 0x55，当前角度数据作为校准数据。
- 数据传输协议，采用 X-Protocol 协议
- 帧定义：AA-55 | 11 | XX | XX-XX XX-XX XX-XX XX-XX XX-XX XX-XX | XX
- 帧解释：帧头 | 帧长 | 帧码 | 滑块 A 滑块 B 滑块 C 滑块 D 滑块 E 滑块 F | 校验和
- 帧长度：17 个字节，固定值 0x11，包括 5 个帧控制位和 12 个数据位
- 帧编码：可用户自定义，默认 0x60(96)，范围 0~255
- 滑块：机械臂 6 个舵机控制帧，范围±1350，单位度，放大 10 倍
- 校验和：前面数据累加和的低 8 位

2.5. 协议发送

- 实现 8 个有符号 16 位数据的发送功能，数据范围-32767~32767，主要用于命令控制和参数调试，例如 PID 参数调试。

- 数据传输协议，采用 X-Protocol 协议

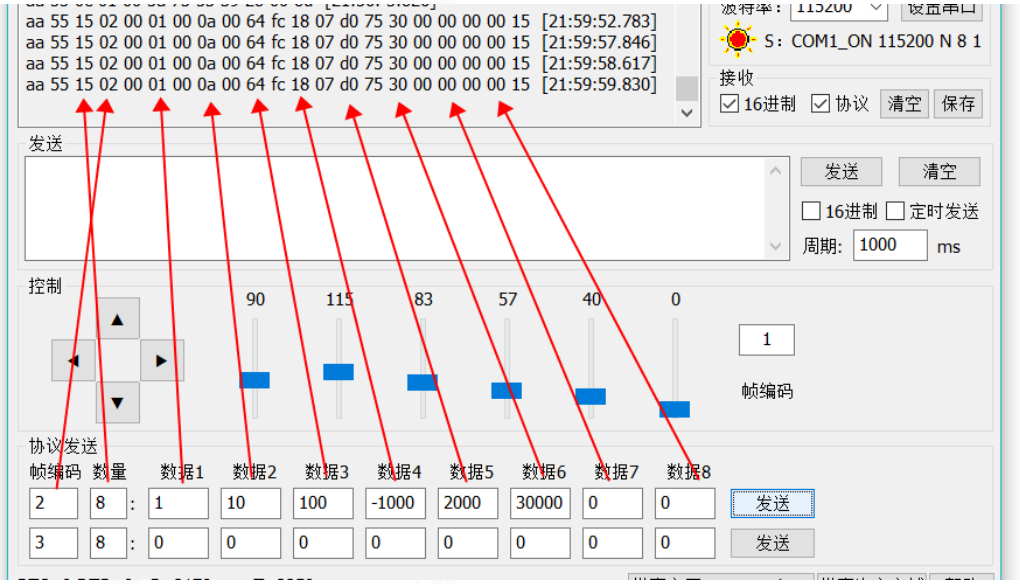
- 帧长度：根据传输数据的数量自动计算

- 帧编码：可用户自定义，默认 0x02，范围 0~255

- 数据：数据内容为用户根据需要填写，范围-32767~32767,16 位有符号型，根据前面定义数据数量发送，例如数量为 2，则发送数据 1 和数据 2。

- 校验和：前面数据累加和的低 8 位

发送数据示例：（采用串口自发自收回环测试）



- 可以在发送区，发送您发送的任意字符。

- 支持 16 进制发送，勾选“16 进制”复选框后，发送的时候将对发送区的内容进行 16 进制和字符互转。

- 支持定时发送，可以自定义发送周期。

- 清除按钮可以清除发送区的内容，并且清除发送字节计数。

3. 协议说明

3.1. 协议介绍

为了更好的使用串口进行数据传输，塔克创新设计了一个通用的通信传输协议 X-Protocol 协议，方便塔克创新产品使用，包括软件产品和硬件产品。

X-SOFT通用串口传输协议：X-Protocol协议（变帧长）

AA 55 0B 01 00 01 00 02 00 03 55

帧头　帧长度　帧号　数据：高位在前，长度可变，自由组合　校验和

帧头	双帧头，抗干扰强
帧长度	根据数据长度设定
帧号	用户根据功能设定，标识帧的唯一性
数据	高位在前，长度可变，内容自由组合8位，16位，32位数据
校验和	前面数据累加和的低8位

3.2. 参考代码

使用 OpenCTR 控制器的参考接收代码。

```
//接收
static u8 UART_RX_BUF[40];  //接收缓冲，数据内容小于等于32Byte
static u8 UART_RX_CON = 0;  //接收计数器
static u8 UART_RX_CHECKSUM; //帧头部分校验和
/**
* @简　述　串口中断服务程序
* @参　数　无
* @返回值　无
*/
void USART1_IRQHandler(void)
{
    uint8_t Res;

    if (USART_GetITStatus(USART1, USART_IT_RXNE) != RESET)  //接收中断
    {

        Res = USART_ReceiveData(USART1);
```

```
        if (UART_RX_CON < 3)     //==接收帧头 + 长度
        {
            if (UART_RX_CON == 0)  //接收帧头1 0xAA
            {
                if (Res == 0xAA)
                {
                    UART_RX_BUF[0] = Res;
                    UART_RX_CON = 1;
                }
                else
                {

                }
            }
            else if (UART_RX_CON == 1) //接收帧头2 0x55
            {
                if (Res == 0x55)
                {
                    UART_RX_BUF[1] = Res;
                    UART_RX_CON = 2;
                }
                else
                {
                    UART_RX_CON = 0;
                }
            }
            else  //接收数据长度
            {
                //USART1_RX_LEN = ( Res-1 );
                UART_RX_BUF[2] = Res;
                UART_RX_CON = 3;
                UART_RX_CHECKSUM = 0xFF + Res; //计算校验和，0xFF为0xAA、0x55校验和
            }
        }
        else    //==接收数据
        {
            if (UART_RX_CON < (UART_RX_BUF[2] - 1))
            {
                UART_RX_BUF[UART_RX_CON] = Res;
                UART_RX_CON++;
                UART_RX_CHECKSUM = UART_RX_CHECKSUM + Res;
            }
            else    //判断最后1位
```

```
            {
                //由于没有配套软件，暂时无需校验
                if (Res == UART_RX_CHECKSUM)  //校验正确
                {
                    //此处进行数据解析
                    UART_Unpack();
                }
                else    //校验错误
                {
                    //不处理，输出错误提示

                }

                //接收完成，恢复初始状态
                UART_RX_CON = 0;
            }
        }
        USART_ITConfig(USART1, USART_IT_RXNE, ENABLE);
    }
}
```

4. 其它说明

暂无。